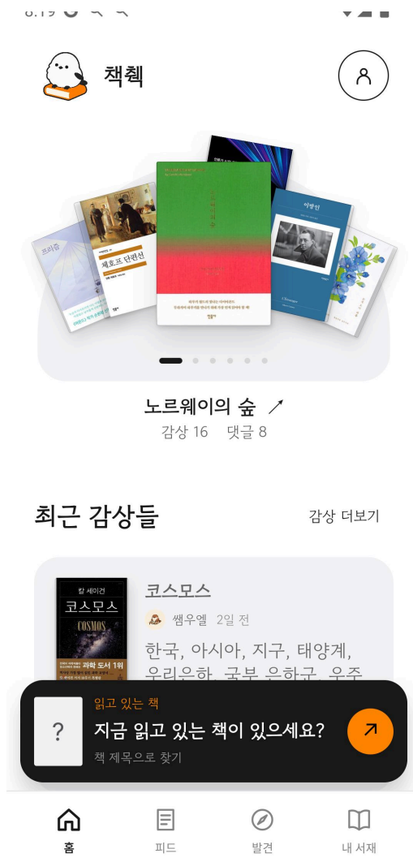
책췍 github.com/woowacourse-teams/2026-chaekchaek

사용자 실험으로 감상 작성 피로도 확인

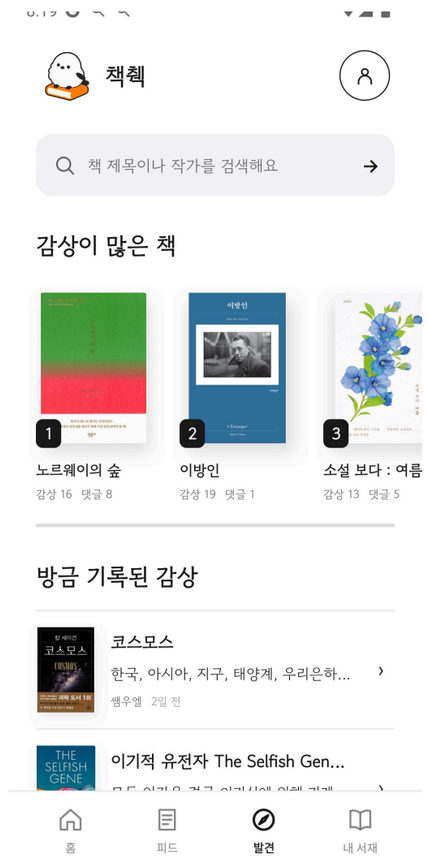
우아한테크코스 팀 프로젝트 / 2026.08 - 현재
실험 설계, AI 활용 구현, 홍보, 인터뷰, 결과 해석

책췍은 읽은 책의 감상을 익명 또는 닉네임으로 남기고 다른 사람과 나누는 서비스입니다.

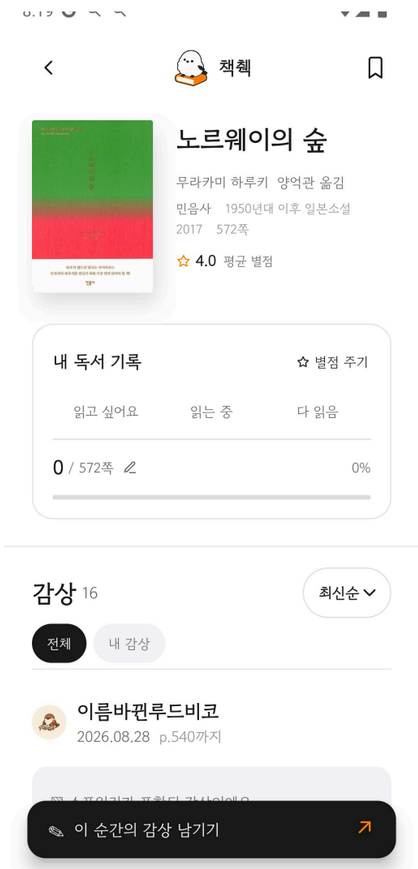
사람들이 어떤 조건에서 감상을 남기는지 확인하기 위해 실험을 설계하고 Codex로 웹 페이지를 구현했습니다. SNS와 커뮤니티에 직접 홍보하고, 참여 기록과 인터뷰를 바탕으로 다음 실험의 질문과 참여 방식을 정했습니다.



홈 화면



발견 화면



도서 상세 화면



도서 상세 화면 - 감상

1차 실험

조회 대비 낮은 감상 참여 확인

어떤 장르에서 감상을 나누고 싶어 하는지 보려고 Codex로 하루 안에 실험 페이지를 만들어 여러 장르의 책을 올렸습니다. 조회는 많았지만 감상을 남기지 않거나 감상으로 보기 어려운 댓글이 달리는 등 참여율이 낮았습니다.

원인 분석

실험 방법, 데이터, 배포 채널 점검

책 선정, 장르에 대한 관심, 작성 부담 중 무엇이 참여를 막았는지 구분하기 어려웠습니다. 실험 방법, 데이터, 배포 채널을 점검하고 팀 회의와 피드백을 거쳐, '감상을 쓸 맥락이 부족했을 가능성'을 다음 실험의 가설로 정했습니다.

2차 실험

줄거리, 발췌문, 기존 감상으로 작성 맥락 제공

책 두 권에 집중해 발췌문 6쪽과 기존 감상을 제공하고, 이방인에는 앞선 줄거리도 추가했습니다. 맥락이 노출된 뒤 작성으로 이어지는지 확인하기 위해 열람, 화면 노출 시간, 작성 시작과 제출을 기록했습니다. 읽음 여부와 유입 경로, 기기는 반응의 차이를 해석하기 위한 기준으로 수집했습니다.

결과 해석

맥락 제공 이후에도 낮은 미독서자 참여 확인

맥락을 제공한 뒤에도 책을 읽지 않은 참여자의 감상 제출은 드물었습니다. 발췌문 열람도 다음 쪽으로 충분히 이어지지 않았습니다. 같은 유입 채널에서도 책에 따라 읽음 여부의 분포가 달라, 채널 전체의 특성으로 일반화하지 않았습니다. 이 결과만으로 원인을 확정하지 않고 작성 부담에 관한 의견을 3차 실험에 반영했습니다.

3차 실험

참여 방식 단순화와 독자 간 상호작용 확인

감상 작성 자체가 피로하다는 의견을 받아 이모지와 선택형 질문으로 참여하고, 원하면 150자 이내의 한 줄 감상을 남기는 실험을 공개했습니다. 참여 방식별 반응과 타인 감상 노출 이후의 좋아요, 답글을 기록했습니다. 제출 여부와 감상 내용의 유효성을 구분해 결과를 해석했습니다.

사용 기술

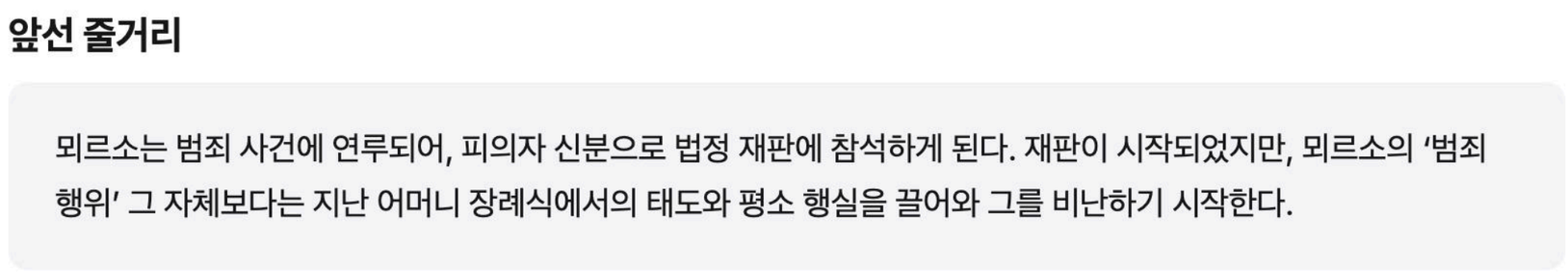
Kotlin / Jetpack Compose / Compose Multiplatform

책췍 실험 화면과 결과

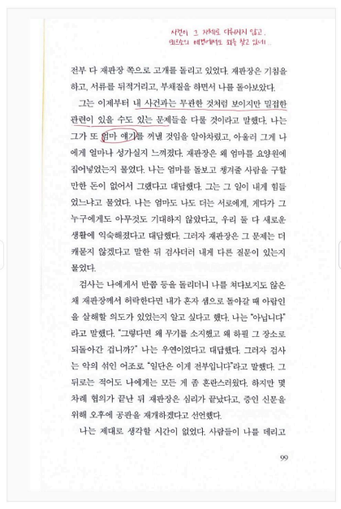
작성 맥락 제공과 참여 방식 단순화

2차 실험 - 줄거리, 발췌문, 기존 감상으로 작성 맥락 제공

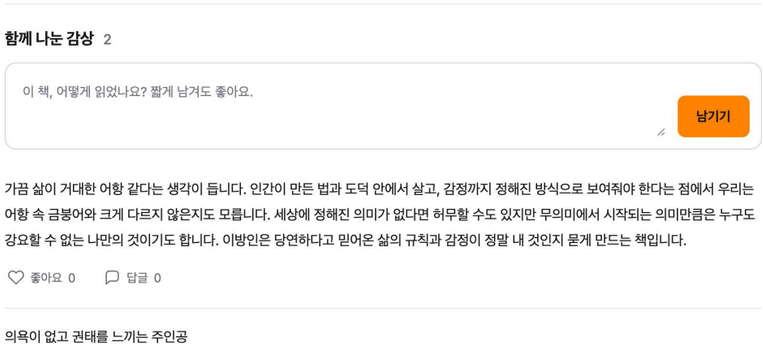
1. 앞선 줄거리와 상황 제시



2. 발췌 이미지 6쪽 제공



3. 감상 작성 공간



2차 실험 집계 결과

이방인, 선택 ID 56개 중 제출 ID 2개 (3.6%)
사랑의 단상, 선택 ID 29개 중 제출 ID 2개 (6.9%)

3차 실험 선택형 반응과 선택 사항인 한 줄 감상

1. 이모지와 선택형 질문



2. 한 줄 감상과 타인 감상 진입



3차 실험 집계 결과 (2026.10.06 - 2026.10.11)

책 페이지 방문 ID 60개, 이모지 또는 투표 참여 ID 16개, 한 줄 제출 ID 21개, 타인 감상 노출 ID 37개, 좋아요 또는 답글 참여 ID 3개

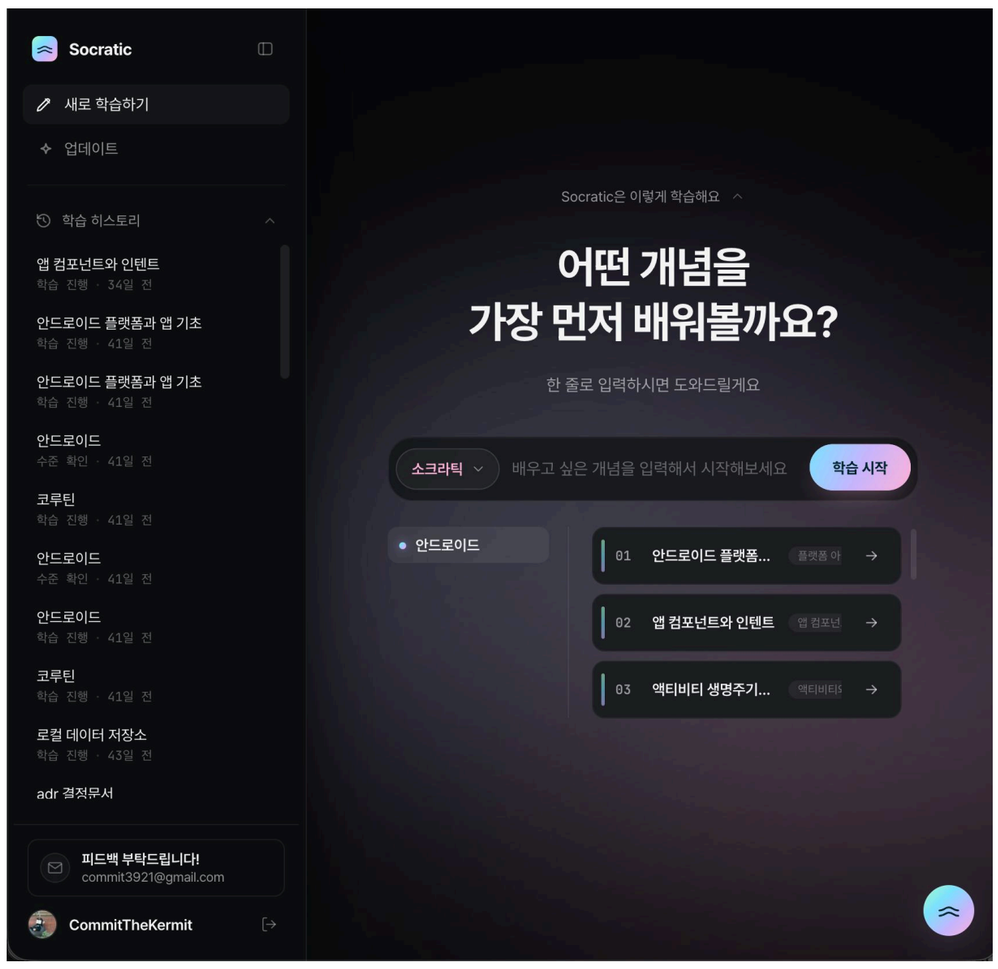
소크라틱 런 socratic-learn-web.web.app

로그인 전에 도망가는 사람들, 학습 질문만 필터

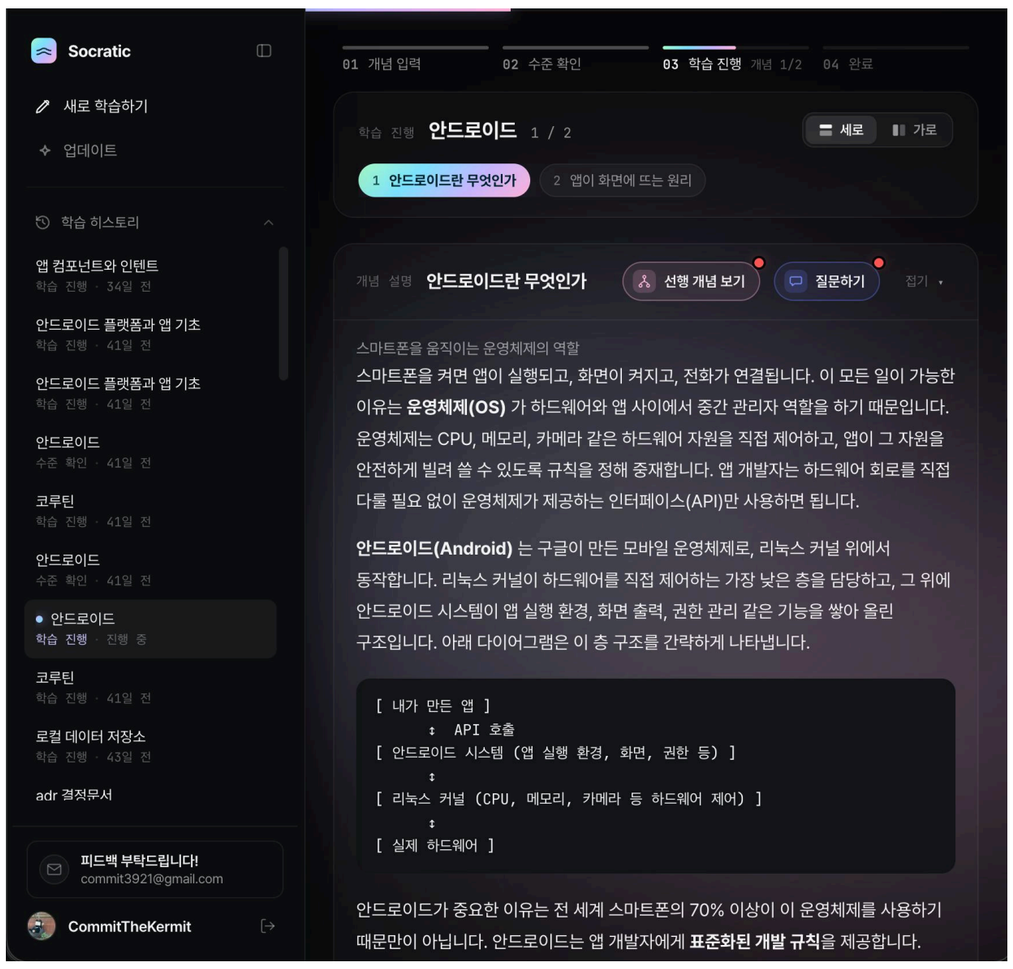
4명 해커톤 팀 개발 후 개인 운영 / 2026.05 - 현재
웹 서비스 운영, 피드백 기반 개선

배우고 싶은 주제를 입력하면 설명을 쏟아내는 대신 질문부터 던집니다. 진단 문항으로 아는 것과 모르는 것의 경계를 찾고, 주제를 작은 개념으로 나눠 배울 순서를 그린 다음, 한 개념씩 직접 답하며 이해를 확인합니다. 답변을 채점해 복습, 선행 학습, 다음 단계, 심화 중 지금 필요한 길을 추천합니다.

코루틴 사용 근거를 묻는 코드 리뷰를 계기로, 설명 뒤 확인 질문에 답하며 이해를 점검하는 Claude Code 스킬을 만들었습니다. 동료에게 공유한 뒤 스킬은 편리하지만 CLI에서 사용하기 불편하다는 반응을 받아 해커톤에서 웹 서비스를 제안했습니다. 4명 팀으로 개발한 뒤에는 혼자 운영하며 피드백과 오류를 개선하고 있습니다.



메인 화면



학습 화면

문제	**로그인 제한 기간의 학습 진입률 11.5% 확인** AI 호출 비용을 제가 부담해야 해서, 무제한 호출이 걱정돼 로그인한 사용자만 질문하게 막았습니다. 계산이 아니라 불안이 근거였습니다. GA4 퍼널로 17일을 보니 학습 진입은 11.5%였고, 로그인은 제가 세운 장벽이라 가장 먼저 의심할 지점이었습니다.
판단	**로그인 제한을 호출 상한으로 대체** uid 기준 일일 150회·분당 20회 상한을 Anthropic 호출에만 걸고, 카운터는 트랜잭션으로 올려 동시 호출에도 새지 않게 했습니다. 저장소 장애 시에는 통과시키고 최종 방어선은 API 결제 한도에 뒀습니다. 기록 저장은 익명 인증으로 옮기고, 로그인하면 linkWithPopup으로 같은 uid를 계정으로 승격합니다.
구현	**실제 로드맵을 기준으로 학습 선택지 구성** 진단 → 로드맵 → 학습 → 분기 → 완료 흐름을 만들었습니다. '다음 단계로 이동'은 LLM이 없는 로드맵을 지어내서, 프론트가 실제 로드맵으로 선택지를 만들게 했습니다. 자주 찾는 주제는 난이도별 로드맵을 미리 두었고(Android 로드맵은 직접 작성), 주요 흐름은 단위 테스트와 Playwright로 묶었습니다.
결과	**변경 전후 학습 진입률 11.5%, 26.7% 관측** 변경 후 학습 진입률은 26.7%로 관측됐지만 방문자는 15명이었습니다. 로그인 화면 이벤트가 없어 page_view에서 학습 시작까지를 대체 지표로 사용했습니다. 표본이 작고 로그인 단계의 이탈을 직접 측정하지 못해, 로그인 제한 해제의 효과로 단정하지 않았습니다.
운영 개선	**사용자 피드백 반영과 학습 진행 오류 수정** 공개 후 접수한 콘텐츠 중복, 혼란스러운 번호 체계, 수식 표시 문제에 대응해 중복 단계 생성을 억제하고 번호 표시를 정리했습니다. 미니 로드맵과 진행 바, KaTeX 수식 렌더링도 적용했습니다. 별도로 직접 사용하며 발견한 평가·분기 건너뛰기와 진행 교착을 수정하고, 학습 중 질문하기와 답변 재제출을 추가했습니다.
사용 기술	React / Vite / Firebase (Auth · Firestore · Functions · Hosting) / Claude API / Vitest / Playwright

주식회사 네모반듯, 툭 tuk.im

너무 뜨거운 앱, 포인트를 퍼다주는 앱

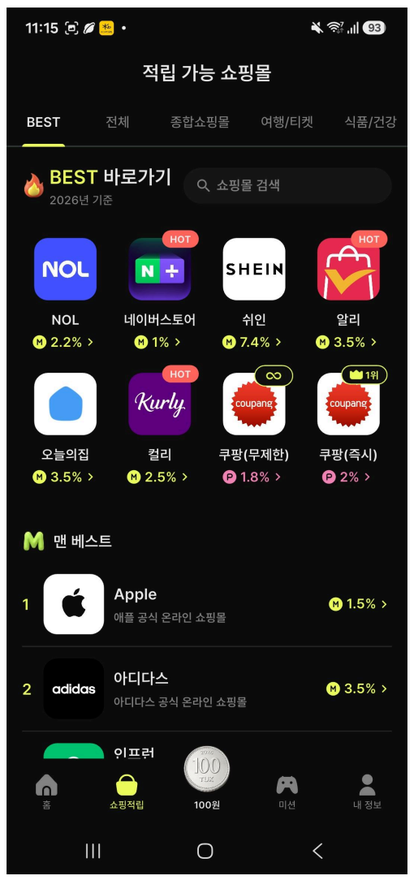
주식회사 네모반듯 / 2025.09 - 2026.02
단독 개발과 운영

툭은 제휴 쇼핑몰에서 구매하면 캐시백을, 미션과 참여형 콘텐츠로 포인트를 적립해 출금하는 리워드 커머스 앱입니다.

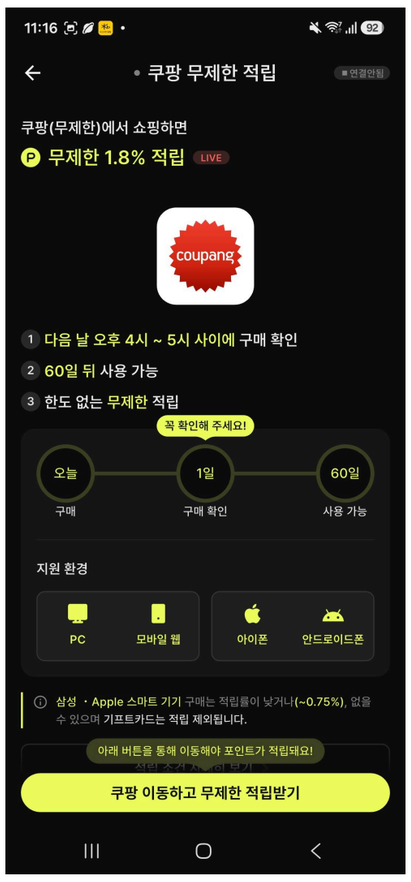
9월에 입사하자마자 외주 개발자에게 인수인계를 받아, 외주사가 6개월간 개발한 앱을 혼자 이어 개발하고 운영했습니다.



메인 화면



쇼핑몰 목록 화면



쇼핑몰 상세 안내

발열

손에 쥐기 뜨거운 앱, 화면의 부담을 하나씩 줄였습니다

앱을 켜 두면 손으로 느낄 만큼 발열이 심했습니다. 과도한 동적 요소부터 줄이고, 캐러셀 자동 순회, 높은 이미지 해상도, 동영상 용량과 화질, 자동 재생을 차례로 줄였습니다. 여러 요소의 부담을 낮춘 결과 체감 발열이 줄었습니다. 온도나 자원 사용량을 수치로 측정하지는 않아 특정 요소 하나를 원인으로 단정하지 않았습니다.

재가입 보상

가입 보상 반복 지급 차단과 지급분 환수

출납 모니터링 중 탈퇴와 재가입을 반복하면 가입 보상 500포인트가 계속 지급되는 문제를 발견했습니다. 본인인증 식별값이 유지되지 않아 같은 사람을 새 사용자로 처리하고 있었습니다. 식별값이 유지되도록 수정해 보상 반복 지급을 막고, 반복 지급된 포인트를 환수했습니다.

회고

기능 추가 시 발열 상태 확인

이 일을 겪은 뒤에는 기능이 동작하는 것뿐 아니라, 실행 중 발열이 어떻게 달라지는지도 살피며 개발했습니다. 특히 움직임이나 미디어를 추가할 때는 화면 효과와 함께 기기에 주는 부담을 확인하며 조심스럽게 적용했습니다.

개발과 운영

앱과 백엔드 개발, 운영 환경 유지보수

AI 도구를 활용해 Flutter 앱과 Spring 백엔드의 기능을 개발하고, 네이버 클라우드의 Kubernetes 운영 환경과 PostgreSQL을 유지보수했습니다.

사용 기술

Flutter / Dart / Riverpod / flutter_hooks / Firebase / Spring / Kubernetes / PostgreSQL

뉴지니(헤스티아, 비드톡톡) play.google.com/store/apps/details?id=kr.co.newjini.bidtoktok

하나의 코드베이스로 협력사 브랜드 앱 14개 배포

뉴지니 / 2024.01 - 2025.08
Flutter 앱 개발, 구조 설계, 빌드, 배포

비드톡톡 / 2024.01 - 2024.07
전자입찰 공고 탐색과 AI 분석을 제공하는 앱입니다. Flutter로 개발해 Android와 iOS 스토어에 출시했습니다.

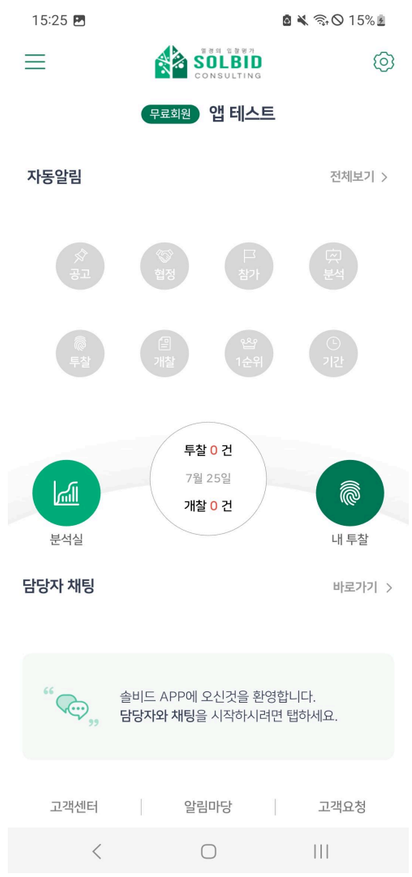
헤스티아 / 2024.07 - 2025.08
전자입찰 공고 탐색과 컨설팅 분석을 제공하는 앱입니다. 기존 Kotlin/XML 앱을 Flutter로 포팅하고, 협력사 14곳의 브랜드 앱을 공통 코드와 브랜드별 빌드 설정으로 관리했습니다.



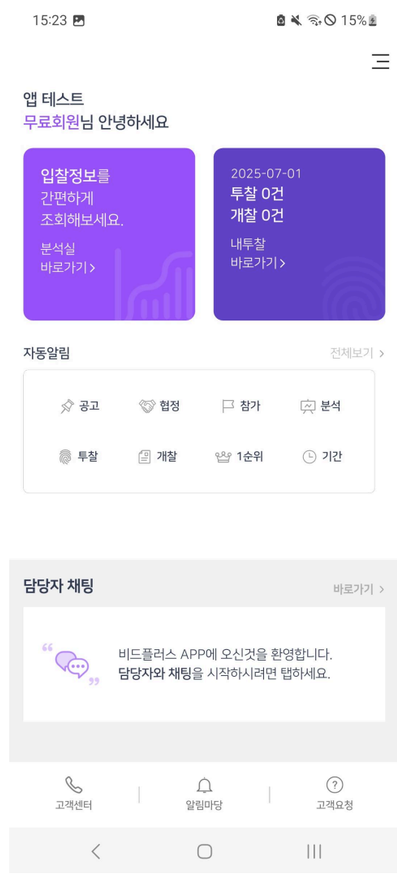
진비드 메인화면



비드윈 메인화면



솔비드 메인화면



비드플러스 메인화면

문제

협력사 증가에 따른 브랜드별 앱 배포 필요

브랜드마다 다른 것이 앱 이름과 아이콘만이 아니었습니다. 협력사가 각자 자기 도메인으로 운영해 API 서버 주소가 달랐고, 음성 안내 파일과 Firebase 프로젝트도 브랜드별로 따로였습니다. 코드를 복사해 앱을 따로 만들면 기능 하나를 고칠 때마다 같은 수정을 브랜드 수만큼 반복해야 했습니다.

구조

빌드 설정을 통한 브랜드별 앱 분리

공통 코드에 브랜드별 빌드 설정을 분리해 앱 이름, 패키지명, 서버 주소, Firebase 설정과 서명 설정을 관리했습니다. 각 앱에는 자기 브랜드 설정만 들어가고, 브랜드가 늘어도 코드베이스를 복사하지 않습니다. 브랜드 추가 절차는 문서로 정리하고 런처 아이콘은 스크립트로 한 번에 생성했습니다.

운영

2주간 브랜드 앱 14개 추가, 총 16개 앱 운영

2024년 7월 공통 구조를 마련한 뒤, 그해 10월에는 2주간 브랜드 앱 14개를 추가했습니다. 서명 키 3종과 브랜드별 Firebase 설정을 관리했고, FCM data 메시지로 받은 알림은 앱에서 구성해 종류별로 TTS 음성과 아이콘을 적용했습니다. 모바일 담당이 한 명으로 줄어든 뒤에도 브랜드 앱 14개, 비드톡톡 1개, 관리자·인트라넷 앱 1개로 총 16개 앱의 배포와 스토어 심사를 운영했습니다.

사용 기술

Flutter / Dart / Firebase / Android Gradle / Spring / MariaDB